LABORATORIUM 4. UKŁADY PERYFERYJNE MIKROKONTROLERA AVR. PORTY. OBSŁUGA KLAWIATURY MATRYCOWEJ

Cel laboratorium:

Zapoznanie studentów z budową i sposobem programowania układów we/wy mikrokontrolera z rdzeniem AVR:

- **- konfiguracja portów do obsługi urządzeń zewnętrznych,**
- **- obsługa klawiatury przeglądanej sekwencyjnie.**

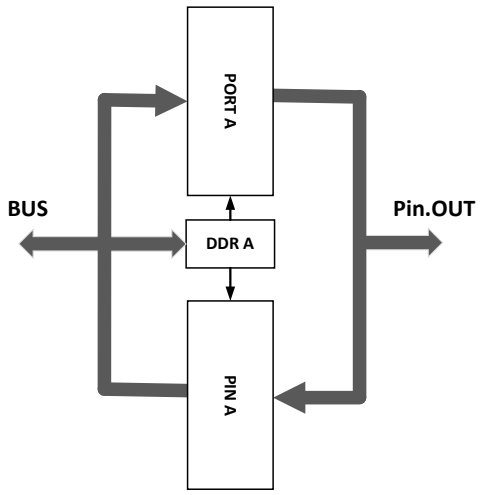
Zakres tematyczny zajęć:

zakres 1, Porty mikrokontrolera ATmega32.

Mikrokontroler ATmega32 zawiera 64 rejestry specjalne (rejestry I/O) i 32 rejestry uniwersalne (Rx). Nazwy rejestrów specjalnych są związane z funkcją jaką te rejestry pełnią, bądź ze sprzętem którego pracę konfigurują (np. rejestr statusu SREG, rejestry DDRx, PORTx, PINx portów I/O,). Natomiast nazwy rejestrów uniwersalnych składają się z numeru rejestru (od 0 do 31) poprzedzonych literą R (np. R16). Rejestry uniwersalne nie są dostępne z poziomu języka C. Dostęp do nich możliwy jest poprzez wstawki asemblerowe. Aby móc korzystać ze zdefiniowanych nazw rejestrów należy program rozpocząć dyrektywą:

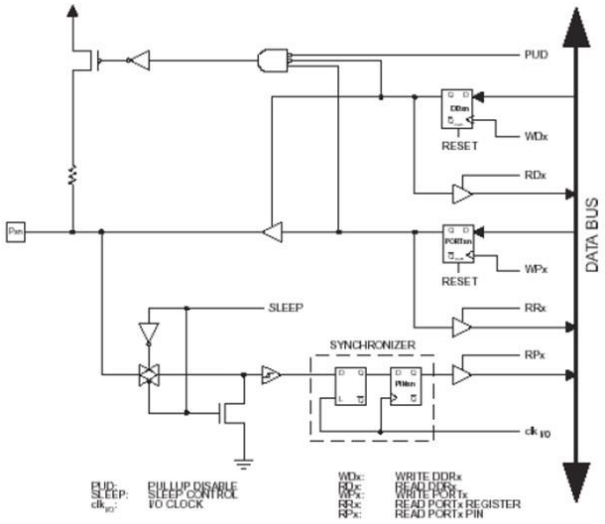
```
#include <avr/io.h>
```

Rejestry specjalne służą do konfigurowania i sterowania pracą peryferii mikrokontrolera (np. portów, liczników, przetwornika analogowo-cyfrowego, itd.). Jednym z podstawowych peryferii każdego mikrokontrolera są porty wejścia/wyjścia. Służą one między innymi do wysyłania i odbierania danych przez MCU. Mikrokontroler ATmega32 wyposażony jest w 4 porty (A, B, C, D). Do konfiguracji kierunku portu (wejście/wyjście) służy rejestr specjalny DDRx (Data Direction Register, gdzie x to nazwa portu np. dla portu A rejestr będzie nosił nazwę DDRA).



Rys. 4.1. Struktura logiczna portu mikrokontrolera

Jak wynika z funkcji portu do odczytu stanu linii we/wy (Pin.OUT) służy rejestr PINx. Rejestr ten odczytuje stan logiczny będący funkcją wyjścia rejestr PORTx oraz stanów logicznych na liniach we/wy uzależnionych od cyfrowych układów zewnętrznych dołączonych do tych linii.



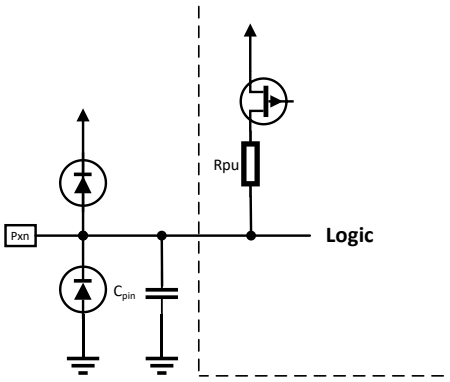
Rys. 4.2. Schemat ideowy portu mikrokontrolera

Tabela 2.1 Konfiguracja wyprowadzeń

DDRn	PORTn	PUD	I/O	pull-up	STAN
0	0	X	In	No	Tri - state
0	1	0	In	Yes	In - Low
0	1	1	In	No	Tri - state
1	0	X	Out	No	Out Low
1	1	X	Out	No	Out High

PUD – bit w rejestrze SFIOR. Umożliwia blokowanie funkcji pull-up dla wszystkich portów.

Każdemu z bitów w rejestrze DDRx odpowiada fizyczna nóżka (pin-out) wejścia/wyjścia mikrokontrolera, znajdująca się na porcie X. Np. bit nr 3 w rejestrze DDRC (czyli PC3) odpowiada nóżce podpisanej „Port C 3” na makiecie dydaktycznej.



Rys. 4.3. Struktura pin-outu portu mikrokontrolera

Końcówki są zabezpieczone przed przepięciami (diody ESD). Istnieje możliwość aktywacji rezystora podciągającego do VCC (pull-up).

Aby skonfigurować daną nóżkę jako wyjście należy wpisać w odpowiednie miejsce w rejestrze DDRC wartość logiczną ′1′. Jeżeli wpisana zostanie wartość logiczna ′0′ to dana linia portu będzie skonfigurowana jako wejście. Do wpisywania wartości do rejestrów służy między innymi instrukcja:

```
DDRA=0xFF;
```

Użyta powyżej instrukcja powoduje przepisanie wartości do rejestru DDRA. Wykonanie tego fragmentu kodu spowoduje wysterowanie wszystkich 8 nóżek portu A (PA0..7) jako wyjścia.

Możliwe jest skonfigurowanie części nóżek portu jako wejścia, części jako wyjścia, np.:

```
DDRA=0x0f;   //konfiguracja wyprowadzeń portu A jako wejścia i
wyjścia
```

Taki zapis spowoduje, że 4 najstarsze nóżki portu A (linie PA7..5) będą wejściami, natomiast pozostałe będą wyjściami (linie PA4..0).

Jeżeli dana nóżka (nóżki, cały port) pracuje jako wejście stany logiczne odpowiadające sygnałom dostarczonym z zewnątrz do linii portu są wpisywane przez mikrokontroler do rejestru PINx. Aby odczytać wartość z portu i zapisać ją do rejestru uniwersalnego należy użyć instrukcji:

```
char x=PINA;
```

Jeżeli dana nóżka (nóżki, cały port) pracuje jako wyjście to wysyłane nią dane należy wpisywać do rejestru PORTx. W wypadku wysyłania danych np. portem A należy użyć rejestru PORTA (uwaga: port A" oznacza jedno z urządzeń wewnątrz mikrokontrolera, „PORTA" oznacza jeden z rejestrów sterujących portem A):

```
PORTA=0b01010101;
```

Wysłanie informacji 0b01010101 przy użyciu portu A.

```
DDRA=0x00;
PORTA=0xFF;
```

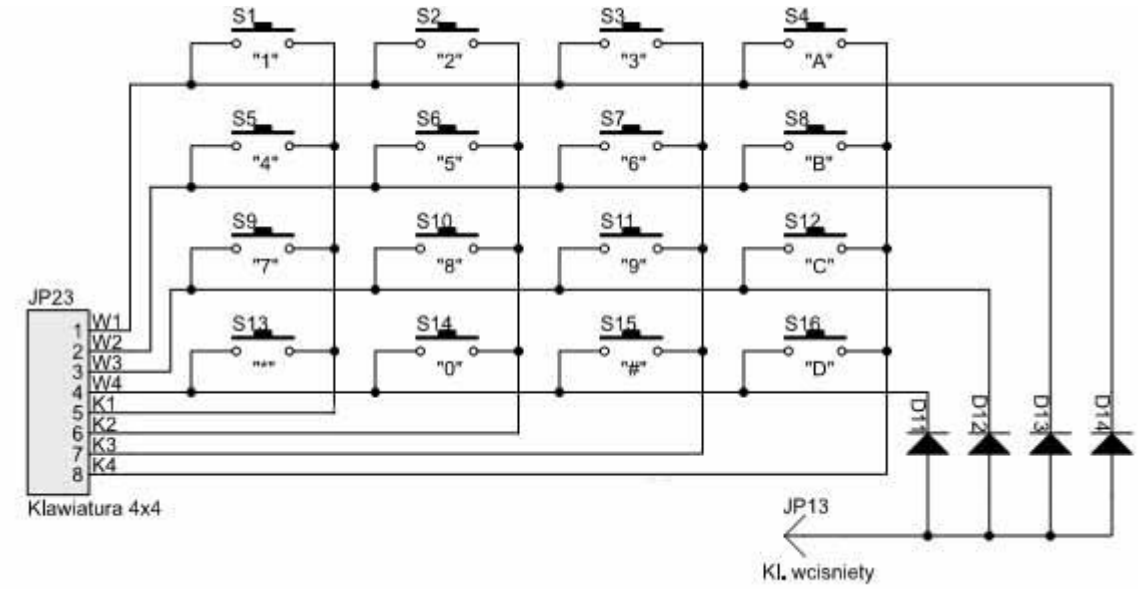
Instrukcja ta przy skonfigurowaniu portu jako wejście powoduje podciągnięcie do „1" wejścia portu. W stanie gdy do linii nie są podłączone zewnętrzne sygnały, odczytywany stan portu będzie wynosił: PINA=0xFF;

Wpisanie powyższego programu do mikrokontrolera i podłączenie jego portu A z wejściami sterującymi pracą diod LED – Złącze JP22 spowoduje, że zaświeci się co druga dioda (D1, D3, D5, D7), co druga nie będzie świeciła (D2, D4, D6, D8).
Podsumowując:

- mikrokontroler ATmega32 posiada 4 urządzenia typu 'PORT': portA, portB, portC, portD;
- do sterowania każdym z tych urządzeń służą 3 rejestry specjalne: DDRx, PORTx i PINx;
- bity w tych rejestrach oznaczamy w jednolity sposób: PC0, PB2, PA7, PD5, itp.

Zakres 2, budowa klawiatury, czytanie klawiatury w układzie matrycowym

Nieodzownym elementem każdego systemu mikroprocesorowego jest klawiatura. Umożliwia ona wpływ użytkownika na wykonywany przez niego program. Jednak teoretycznie zastosowanie w systemie klawiatury składającej się z 16-tu przycisków wymaga zastosowania takiej samej ilości linii portu wejścia-wyjścia.



Rys. 4.4. Schemat klawiatury matrycowej zestawu ZL3AVR

Zastosowanie matrycy klawiaturowej pokazanej na rysunku 4.4 pozwala na zredukowanie ilości linii portu do ośmiu. Cała operacja wymaga jednak napisania odpowiedniego programu tzw. przeglądania klawiatury. Jak widać mamy w tej klawiaturze cztery kolumny i cztery wiersze. Wiersze mogą być podłączone np.: do linii do D0-D3, kolumny do linii D4-D7 dowolnego portu mikrokontrolera. Dla układu ATMEGA32 korzystając z portu A będą to linie PA0-PA3 (wiersze) oraz PA4-PA7 (kolumny). Mikrokontroler będzie wysyłał na kolejne kolumny tzw. krążące zero i czytał za każdym razem stan linii wierszy. Jeśli w danej kolumnie któryś z klawiszy jest wciśnięty to na danej temu klawiszowi linii wiersza pojawi się stan logiczny „0". Następnie zero przesuwane jest na bit sterujący następną kolumną i znowu następuje odczyt stanu linii wierszy. Osobny problem stanowi eliminacja drgań zestyków, realizuje się ją programowo przez dwukrotne sprawdzanie czy po upływie około kilkunastu mS ten sam klawisz jest nadal wciśnięty, dopiero teraz program może potraktować informację o klawiaturze jako wiarygodną.

Konfiguracja portu do obsługi klawiatury:

#define F_CPU 1000000L

#include <avr/io.h>

#include <util/delay.h>

Podłączenie klawiatury prezentuje się w następujący sposób:

k4	k3	k2	k1	w4	w3	w2	w1
PA7	PA6	PA5	PA4	PA3	PA2	PA1	PA0

```
 void init(void)
        {
            DDRA = 0xF0; //PA0...PA3 wejścia, PA4...PA7 wyjścia
           PORTA = 0x0F; //PA0...PA3 wejścia podciągnięte, PA4...PA7 wyjścia stan niski (0)
        }
```

Skrypt obsługi klawiatury z wykorzystaniem instrukcji sterującej *switch():*

```
uint8_t x,i;
 spr=0;nr=0;
for(i=4;i<8;i++){
PORTA= ~_BV(i); // wprowadź 0 na i-tej pozycji i 1 na innych pozycjach
_delay_ms(1);
x=PINA&0x0F;    // odczytaj i zapamiętaj stan klawiatury
 switch(x)
{
      case 0b00001111:  PORTB=0;   break; // żaden przycisk nie jest wciśnięty
      case 0b00001110:  PORTB = 1; // wiersz 1,  zaświecenie zerowej diody
      break;
      case 0b00001101:  PORTB=5;    // wiersz 2 ….
      break;
      case 0b00001011:  PORTB=9;    // wiersz 3  ….
      ……
      default: brak;
}
```

Należy zwrócić uwagę na sposób przekazywania stanu klawiatury poprzez zmienną x do funkcji *switch* – próbkujemy stan klawiatury uniezależniając się od zmian stanu rejestru PINA w trakcie wykonywania procedury czytania klawiatury.

Pytania kontrolne:

Pytanie 1. Jak należy skonfigurować port we/wy do obsługi klawiatury matrycowej.
Pytanie 2. Jak określić wartość parametru *case* funkcji *switch* do odczytu dwóch jednocześnie naciśniętych przycisków w tej samej kolumnie.

Zadanie 4.1. Programowanie klawiatury – budowa funkcji getkey()

Napisz funkcję getkey() służącą do odczytu klawiatury matrycowej zwracającej:

- kod 0 gdy żaden z przycisków nie jest wciśnięty,

- nr naciśniętego przycisku gdy został dokładnie jeden naciśnięty,

- kod 0xff gdy wciśnięto więcej niż jeden w dowolnej kolumnie lub wierszu.

Funkcę przetestować w oparciu o zestaw ZL3AVR binarnie wyświetlając zwracany kod przez funkcję getkey() na linijce diodowej dołączonej do kolejnego portu, np.: B lub D.

Zadanie 4.2

Napisać program symulujący działanie kalkulatora czterodziałaniowego umożliwiający wprowadzenie dwóch liczb całkowitych i wybór rodzaju operacji na nich. Do wyświetlania wyniku operacji użyj wyświetlacza LED 7SEG. Przyjmij następujące funkcje przycisków:

Przyciski S0 – S9 - cyfry wprowadzanej liczby

Przycisk: S16 – operacja „=”

Przycisk: S13 – operacja „+”

Przycisk: S14 – operacja „-”

Przycisk: S15 – operacja „x” (mnożenie)

Przycisk: S12 – operacja „/” (dzielenie).